big data and cognitive computing

MDPI

Article

A Real CVE-Backed Benchmark for WordPress Plugin Vulnerability Detection: Re-Evaluating Static and Learning-Based Detectors Under Leakage-Controlled Evaluation

Zhuldyz Tashenova [1], Aisultan Aitmagambetuly [1], Altyn Urynbassarova [1], Askhat Zhetkerbay [1,*], Shirin Amanzholova [2,*] and Akerke Kerim [1]

[1] Department of Information Security, L. N. Gumilyov Eurasian National University, Astana 010000, Kazakhstan; zhuldyz_tm@mail.ru (Z.T.)
[2] Department of Information Technologies, Kurmangazy Kazakh National Conservatory, Almaty 050000, Kazakhstan
* Correspondence: askhatzh9898@gmail.com (A.Z.); sh.amanzholova@gmail.com (S.A.)

Abstract

Background: WordPress plugins account for the large majority of disclosed CMS vulnerabilities, and learning-based detectors report high accuracy on synthetic corpora and random splits. Methods: We build a benchmark from 1757 real plugin CVEs (6666 indexed identifiers, 1281 plugins), yielding 30,860 labeled PHP functions in two negative-sampling variants. An audit of the released artifacts found exact normalized-code duplicates crossing test folds (166 groups/1086 records in Variant A; 151/676 in Variant B) and identical code carrying contradictory labels (730 and 456 groups). After removing conflicting groups, merging exact duplicates and collapsing token- and AST-level clone families, the corpora contain 15,294 (Variant A) and 24,917 (Variant B) records with zero shared hashes, clone families or plugins across folds. Ten detectors are compared under plugin-grouped five-fold cross-validation on complete held-out folds: a static taint analyzer, TF-IDF Random Forest and SVM, Bi-LSTM, unidirectional LSTM, a taint-augmented neural model, a nested out-of-fold stacking ensemble, and three baselines (majority class, a source/sink presence rule, and a single-feature function-length classifier). Results: The best macro-F1 is 0.563 (Variant A, binary, SVM) and 0.601 (Variant B, binary, nested stacking). Neural models do not lead in any of the six configurations, and the single-feature length baseline matches or exceeds the TF-IDF Random Forest in four of six. The source/sink rule attains an MCC close to zero, and the taint analyzer misses about 98% of vulnerabilities. Manual review of 400 functions finds 48.8% of cleaned positive labels to be genuine vulnerabilities. Conclusions: Under duplicate-controlled evaluation, the differences between detector families are small compared with the variation attributable to label quality, which appears to be the more binding constraint. All data, folds, predictions and fingerprinted result files are released.

Keywords: vulnerability detection; WordPress plugins; PHP static analysis; taint analysis; CVE dataset; leakage-controlled evaluation; duplicate detection; label noise; benchmark

Check for updates

Academic Editor: Francesco Cauteruccio

Received: 3 July 2026
Revised: 14 August 2026
Accepted: 31 August 2026
Published: 4 September 2026

Copyright: © 2026 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license.

1. Introduction

Content Management Systems (CMSs) underpin a large share of the public web, and WordPress alone serves a substantial fraction of all websites. Empirical studies of deployed installations show that the core platform is usually patched promptly, whereas

third-party plugins lag far behind and account for the overwhelming majority of reported vulnerabilities [1,2]. Cross-site scripting (XSS) and SQL injection (SQLi) dominate the reported classes, with a smaller but more severe tail of remote code execution (RCE) [3].

Prevailing defenses are predominantly reactive. Signature-based scanners and web application firewalls act after a vulnerability is public, and vulnerability databases catalog flaws only after disclosure. This motivates proactive detection directly on plugin source code. In earlier work we proposed SentinelCMS [4], a detector combining static taint analysis with a bidirectional LSTM, which reached approximately 93% accuracy on a synthetic corpus of 600 generated snippets.

Whether such results transfer to real, disclosed vulnerabilities is the question this paper addresses. We assemble a benchmark from real plugin CVEs, evaluate a range of detector families on it, and—prompted by an audit of our own released artifacts—pay particular attention to duplication and evaluation leakage, which turned out to affect the measurements substantially.

An audit of the first release showed that grouping cross-validation folds by plugin slug does not prevent exact code duplicates from crossing fold boundaries, because WordPress plugins routinely vendor shared libraries and administrative menus. It also showed that identical functions sometimes carry contradictory class labels, and that a stacking ensemble described as out-of-fold was in fact trained on in-sample predictions. The present version rebuilds the corpus and the entire benchmark with these problems corrected, and quantifies how much they inflated the earlier numbers.

We address four research questions. RQ1 (transfer): does training on real CVE data outperform training on synthetic data? RQ2 (detector families): how do static, classical and neural detectors compare under duplicate-controlled evaluation? RQ3 (negatives and granularity): how do the negative-sampling variant and label granularity affect the results? RQ4 (evaluation validity): how much do duplicate leakage and in-sample stacking inflate reported performance?

This study makes four contributions. (1) A real, duplicate-controlled benchmark of WordPress plugin CVEs with verified zero overlap of code hashes, clone families and plugins across folds. (2) A comparison of ten detectors, including three deliberately trivial baselines, on complete held-out folds. (3) A direct measurement of how much duplicate leakage and in-sample stacking inflate performance. (4) An expert validation of the label pipeline on 400 manually reviewed functions. All artifacts are released with input fingerprints so that any result can be traced to the data from which it was computed.

2. Related Work

2.1. Vulnerabilities in CMS Ecosystems

Large-scale measurements of deployed websites show that outdated third-party components, rather than CMS cores, dominate the exposure surface [1,5]. Industry reports on the WordPress ecosystem quantify the same pattern, with plugins responsible for the large majority of newly disclosed issues each year [2].

2.2. Static Analysis for PHP

Static detection of injection flaws in PHP has a long history, from early taint-tracking tools such as Pixy [6] to graph-based frameworks. Code-property-graph approaches [7] generalize this by combining syntax, control flow and data dependence in a single queryable structure. Such analyzers are precise when the vulnerable data flow is local and syntactically explicit, but degrade on dynamic dispatch, framework callbacks and flows that cross file boundaries—all common in plugin code.

2.3. Learning-Based Vulnerability Detection

VulDeePecker [8] introduced slice-level neural detection; graph neural networks (e.g., Devign [9]) encode structural dependencies; and line-level transformer variants such as LineVul [10] report F1 above 0.90 on Big-Vul under random splits. Most of this work targets C/C++. For PHP, DeepTective [11] combines gated graph networks with sequence models on a corpus assembled from plugin sources.

2.4. Pre-Trained Code Models

CodeBERT [12] provides bimodal pre-training over code and natural language, GraphCodeBERT [13] injects data-flow structure, and instruction-tuned large language models [14] have been applied to vulnerability reasoning. On established benchmarks such as CodeXGLUE [15] these models reach roughly 62% accuracy on defect detection, a level far below the numbers reported on curated corpora.

2.5. Data Quality and Evaluation Validity

A growing body of work questions whether reported gains reflect detection ability or evaluation artifacts. Chakraborty et al. [16] show that models trained on curated datasets degrade sharply on real-world code. Arp et al. [17] catalog recurring methodological pitfalls in machine learning for security, including sampling bias and spurious correlations. Croft et al. [18] measure label accuracy and duplication in widely used vulnerability datasets and find both to be substantial problems, and Ding et al. [19] report that benchmark scores fall considerably once duplicates and unrealistic splits are removed. The present study contributes a domain-specific instance of this critique, with a measured decomposition of duplication, label noise and ensemble leakage in a PHP/CMS corpus.

3. Materials and Methods

3.1. Dataset Construction

Candidate vulnerabilities were indexed from the NVD CVE API v2 [20] and the public CVE list (CVEProject/cvelistV5) [21]. The two sources overlap because the same CVE is frequently listed in both; after removing duplicate identifiers the merged index contains 6666 unique records, which reconciles the 5839 and 853 source counts (26 shared identifiers). Of these, 1757 CVEs across 1281 plugins could be resolved to downloadable pre-fix and post-fix source pairs (1234 of these plugins still carry vulnerable records after the deduplication of Section 3.2); approximately 54% of indexed CVEs were unrecoverable because old releases and SVN tags had been pruned.

Function-level labels were derived by differencing the vulnerable and patched versions with a tree-sitter PHP parser: functions touched by the security fix receive the CVE's vulnerability type, and untouched functions provide negatives. Two negative-sampling variants are retained: Variant A uses the patched twin of each vulnerable function, and Variant B uses functions from the same plugin that were not modified by the fix. Each variant contains 30,860 snippets before deduplication. A per-record provenance manifest (CVE id, source, plugin slug, version, file, function name and code hash) accompanies the release.

The dataset construction and leakage-control procedure used in this study is illustrated in Figure 1. The workflow shows how function-level labels are derived from vulnerable and patched releases, followed by consistency checks, duplicate removal, clone-family auditing, and fold assignment to minimize potential data leakage.

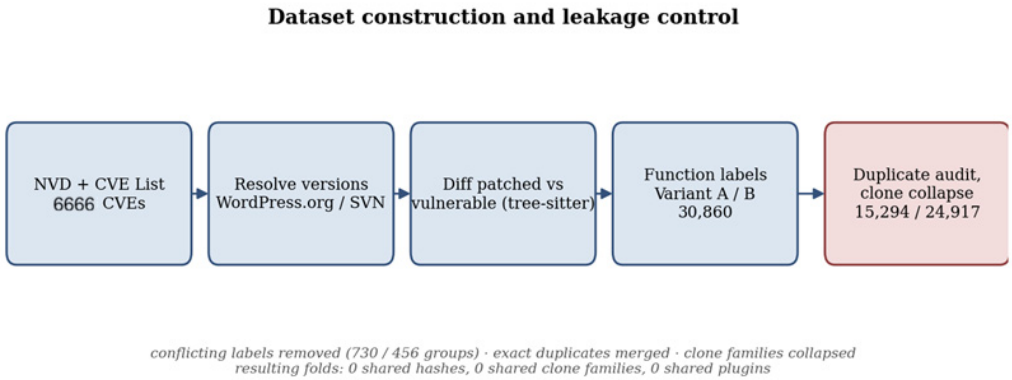


Figure 1. Dataset construction and leakage control. Function-level labels are derived by differencing patched and vulnerable releases; the corpus is then audited for contradictory labels, exact duplicates and clone families before folds are assigned. Blue boxes represent dataset construction steps, and the red box indicates the leakage control stage.

3.2. Duplicate Audit and Leakage Control

Cross-validation folds in the first release partitioned plugin slugs. An audit of the published code_hash fields showed that this is insufficient: identical normalized code appearing under two different plugin slugs is split across training and test folds. Table 1 reports the audit. Beyond exact duplicates, identical code was found carrying contradictory class labels, because diff-based labeling assigns the CVE's type to every changed function, so a function touched by several CVEs receives several types.

Table 1. Duplicate and label-conflict audit of the originally released indices. Exact duplicates are computed on whitespace-normalized code (SHA-256).

Audit Quantity	Variant A	Variant B
Duplicate groups spanning more than one fold	166	151
Records in those groups	1086	676
All duplicate groups (any fold)	2022	983
All duplicated records	5735	2756
Distinct normalized hashes (of 30,860)	27,147	29,087
Groups with contradictory labels	730	456
Records with contradictory labels	1852	1199

Three corrections were applied before any model was retrained. First, every hash group carrying contradictory labels was removed in full: identical code with mutually exclusive labels is unreliable supervision, and retaining either side would inject known-incorrect labels. Second, exact duplicates were merged into a single record. Third, near-duplicates were identified at both the token level (MinHash over 5-token shingles, Jaccard similiarity of at least 0.9) and the AST level (tree-sitter node-type sequences, Jaccard similiarity of at least 0.9) and grouped into clone families.

Grouping folds by the connected components of plugin slug and clone family is leakage-free by construction but produces a giant component: cross-plugin copy-paste, mainly vendored libraries and shared administrative menus, chains most plugins together, so one component covers 59.4% (Variant A) and 72.1% (Variant B) of the corpus and per-fold statistics become meaningless. We therefore collapse each clone family to a single representative record, after which a family exists only once and grouping by plugin slug suffices. Table 2 reports the resulting corpora. The merged-component grouping was therefore rejected rather than adopted, and the decision, together with the component statistics, is recorded in the released collapse report; the collapsed corpora are the ones used throughout and the ones released.

Table 2. Corpora after leakage control. Collapsing near-duplicates removes proportionally more of Variant A, whose patched-twin negatives are highly repetitive; as a consequence, the majority class of Variant A changes from Safe to vulnerable, so accuracy values are not comparable with the previously published version, and macro-F1, balanced accuracy and MCC should be used instead.

Corpus Quantity	Variant A	Variant B
Published records	30,860	30,860
After conflict removal and exact deduplication	26,417	28,631
Near-duplicate records collapsed	11,123	3714
Final records	15,294	24,917
Reduction versus published	50.4%	19.3%
Distinct plugins (total/vulnerability-bearing)	1254/1234	1745/1234
Plugins with both classes/vulnerable only/Safe only	740/494/20	1116/118/511
Class distribution (Safe/XSS/SQLi/RCE)	3895/8446/2104/849	13,396/8531/2142/848
Cross-fold overlap (hash/clone family/plugin)	0/0/0	0/0/0

The two variants share exactly the same vulnerable material: 1234 plugins carry at least one vulnerable record in both, which serves as a consistency check on the rebuild. They differ only in how negatives are drawn, and this is why the plugin totals differ. Variant A takes negatives from the patched twin of each vulnerable function, so almost every plugin contributes both classes (740 of 1254 mixed, only 20 Safe-only). Variant B draws negatives from functions never touched by a security fix, which admits 511 further plugins contributing Safe records only, giving 1745 in total. The distinction also shapes fold composition: 494 Variant A plugins contain vulnerable records exclusively, and since grouping assigns each plugin entirely to one fold, Variant A folds are less balanced across classes than those of Variant B.

3.3. Detectors and Baselines

Ten configurations are evaluated. Static taint analyzer: a two-pass intra-procedural analyzer that propagates taint from user-controlled sources and reports a finding when a tainted value reaches a security-sensitive sink without sanitization. TF-IDF classifiers: Random Forest and a linear-kernel SVM over character n-gram TF-IDF features (char_wb analyzer, 2–4 g over the raw code). Neural models: a Bi-LSTM with attention, a unidirectional ablation, and a combined model that concatenates the taint verdict to the Bi-LSTM representation. Stacking: a logistic-regression meta-learner over base-model predictions, evaluated in two forms described below.

Three baselines are added deliberately because a benchmark is only informative if trivial predictors are shown to be insufficient. The majority baseline always predicts the most frequent training class. The source/sink baseline predicts a vulnerability whenever a taint source and a type-matching sink are both present—this is exactly the rule used to clean labels in Section 4.5, so its standalone detection ability is directly relevant. The function-length baseline is a logistic regression on a single feature, the number of tokens in the function.

The stacking ensemble is reported in two forms. The in-sample form reproduces the originally released implementation, in which the base models are fitted on the complete outer training fold and their predictions on those same records become the meta-features. The nested form generates meta-features strictly out of fold, using an inner StratifiedGroupKFold within each outer training fold, and refits the base models on the full outer training fold only to score the test fold. Only the nested form is citable; the in-sample form is retained solely to quantify the bias in Section 4.4. Both forms use the same base

set; the LSTM tower is omitted from both because nested generation requires twenty-five times more base fits than the available CPU budget allowed, so this row is not directly comparable with the previously published stacking row.

3.4. Experimental Protocol

All models are evaluated with StratifiedGroupKFold (it is the cross-validation splitter from scikit-learn version 1.8.0.) (five folds, seed 42) grouped by plugin slug on the collapsed corpora. Test folds are always the complete held-out set: no balancing or subsampling is applied to any test fold, so all reported metrics are prevalence-preserving. Where the training budget required it, neural models subsample the training side only (at most 4000 records per fold, 40 epochs, sequence length 128); each such decision is recorded in the released sampling manifest. The 128-token bound was chosen for computational tractability on a CPU-only host rather than by tuning, so that the sequence models could be trained across all folds and configurations within the available budget; its effect on the ranking has not been measured, and a 256-token sensitivity analysis is reported as outstanding in Section 5.2. Configurations span the two variants and three granularities: binary (Safe versus vulnerable), three-class (Safe, SQLi, XSS) and four-class (adding RCE).

We report accuracy and macro-F1 as fold means with standard deviation and 95% confidence intervals (t distribution, four degrees of freedom), per-class precision, recall and F1, vulnerable-class recall and false-negative rate, balanced accuracy, Matthews correlation coefficient and PR-AUC. Model comparisons use paired tests both across folds and across plugins, with Cohen's d as effect size; the plugin-level test is the better-powered of the two, since fold-level tests have only five paired observations. Per-fold predictions and class scores are released, so every reported number can be recomputed.

Result files are keyed to input fingerprints. Each artifact records the SHA-256 of the dataset index and of the fold assignment file, and a cached result is reused only if both match the current inputs. This replaces the earlier checkpoint logic, which was keyed on a filename tag and silently reused results computed on an older and smaller corpus.

3.5. Hyperparameters

To ensure reproducibility and facilitate a consistent comparison of the evaluated models, the architectures and training hyperparameters were adopted directly from the released training scripts. The main architectural configurations and hyperparameter settings used in the experiments are summarized in Table 3.

Table 3. Architectures and hyperparameters, taken verbatim from the released training scripts.

Component	Setting
Shared protocol	StratifiedGroupKFold (5 folds), group = plugin_slug, seed = 42; class-weighted cross-entropy with inverse-frequency weights; complete held-out test folds
Bi-LSTM/uni-LSTM	sequence length 128; 40 epochs; embedding 64; hidden 128; 2 layers; dropout 0.3; batch 16; Adam, learning rate 1×10^{-3}; additive attention with LayerNorm; gradient clipping 1.0; at most 4000 training records per fold
TF-IDF features	character n-grams (char_wb), range (2,4); max_features 5000; min_df 1
Random Forest	n_estimators 300; max_depth none; class_weight balanced; random_state 42
SVM	LinearSVC; C 1.0; class_weight balanced; max_iter 2000; random_state 42

Table 3. *Cont.*

Component	Setting
Stacking	logistic-regression meta-learner (class_weight balanced, max_iter 500); nested form uses an inner StratifiedGroupKFold(5) inside each outer training fold
Baselines	majority class; source/sink presence rule; logistic regression on token count
Software	Python 3.14, PyTorch 2.11 (CPU), scikit-learn 1.8, NumPy 2.4, tree-sitter-php; seed 42 throughout

4. Results

4.1. RQ1: Training on Synthetic Versus Real Data

A model trained on the 600-snippet synthetic corpus of the prior work is applied to the entire real corpus and compared with real-to-real cross-validation using the same architecture. Both sides are recomputed on the collapsed corpora; the previously published comparison used a real-to-real value read from a summary file that has since been quarantined as stale. Table 4 reports the outcome.

Table 4. Synthetic-to-real transfer versus real-to-real cross-validation (Bi-LSTM, seed 42). The synthetic model is trained once on 600 generated snippets and tested on the entire real corpus; the real-to-real column is the five-fold grouped cross-validation mean. Δ is real minus synthetic macro-F1.

Setting	Synthetic → Real Acc/Macro-F1	Real → Real Acc/Macro-F1	Δ Macro-F1
Variant A, binary	0.543/0.508	$0.663 \pm 0.046/0.541 \pm 0.059$	+0.034
Variant A, four-class	0.273/0.222	$0.259 \pm 0.049/0.203 \pm 0.035$	−0.019
Variant B, binary	0.494/0.494	$0.576 \pm 0.012/0.572 \pm 0.016$	+0.078
Variant B, four-class	0.286/0.205	$0.307 \pm 0.060/0.188 \pm 0.039$	−0.017

Training on real data yields a clear gain only in the binary setting (+3.4 and +7.8 percentage points of macro-F1). In the four-class setting, the synthetic model is marginally ahead, so roughly twelve thousand real training records provide no advantage over six hundred generated ones. This should be read with care: the synthetic vocabulary contains only 34 tokens, and 0.9% of real tokens fall outside it, so the comparison indicates how little separable signal the fine-grained real labels carry rather than any strength of the synthetic corpus.

4.2. RQ2 and RQ3: Detector Comparison

Table 5 reports macro-F1 for all ten configurations across the six variant-granularity settings, each on the complete held-out folds. Figure 2 shows the two binary settings with the function-length baseline marked.

Table 5. Macro-F1 on complete held-out folds (five-fold plugin-grouped cross-validation, seed 42). Test folds are prevalence-preserving. Test-set sizes: Variant A 15,294 (binary and four-class) and 14,445 (three-class); Variant B 24,917 and 24,069.

Model	A bin.	A 3-cl.	A 4-cl.	B bin.	B 3-cl.	B 4-cl.
stacking (nested)	0.540	0.372	0.268	0.601	0.398	0.281
SVM + TF-IDF	0.563	0.377	0.292	0.562	0.378	0.300
Random Forest + TF-IDF	0.457	0.292	0.196	0.586	0.322	0.231
Bi-LSTM	0.541	0.260	0.203	0.573	0.346	0.188

Table 5. *Cont.*

Model	A bin.	A 3-cl.	A 4-cl.	B bin.	B 3-cl.	B 4-cl.
uni-LSTM	0.526	0.345	0.228	0.578	0.359	0.237
combined (taint + Bi-LSTM)	0.480	0.310	0.161	0.569	0.363	0.234
function length (baseline)	0.446	0.306	0.226	0.572	0.389	0.283
majority (baseline)	0.427	0.246	0.178	0.350	0.238	0.175
source/sink rule (baseline)	0.352	0.198	0.142	0.480	0.278	0.204
static taint analyzer	0.221	0.160	0.116	0.366	0.256	0.188

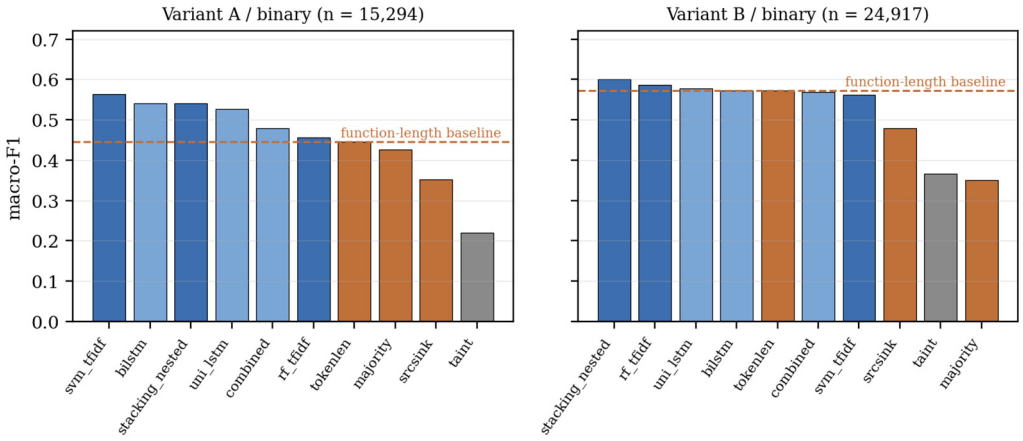


Figure 2. Macro-F1 by model in the two binary settings. The dashed line marks the single-feature function-length baseline.

Three observations follow. First, no neural configuration leads in any of the six settings; the gap to the best classical model ranges from 2.1 to 6.4 percentage points of macro-F1. Second, the single-feature function-length baseline matches or exceeds the TF-IDF Random Forest in four of six settings and is within one percentage point of the best neural model in Variant B binary (0.572 against 0.573), which suggests that much of what the models exploit is correlated with function size rather than with security-relevant structure. Third, the source/sink rule, which is the same criterion used for label cleaning, is not a detector on its own: its MCC ranges from −0.001 to 0.063.

The detailed performance metrics obtained for Variant B under the binary classification setting are presented in Table 6.

Table 6. Detailed metrics for Variant B, binary (n = 24,917). Confidence intervals are computed over the five folds.

Model	Accuracy	Macro-F1 (95% CI)	Bal. Acc.	MCC	Vuln. FNR	PR-AUC
stacking (nested)	0.605	0.601 (0.579–0.624)	0.601	0.203	0.455	0.590
Random Forest + TF-IDF	0.608	0.586 (0.574–0.598)	0.594	0.203	0.593	0.600
uni-LSTM	0.583	0.578 (0.563–0.594)	0.584	0.168	0.401	0.555
Bi-LSTM	0.576	0.573 (0.550–0.595)	0.581	0.162	0.356	0.558
function length (baseline)	0.596	0.572 (0.551–0.593)	0.582	0.177	0.610	0.575
combined	0.579	0.569 (0.530–0.609)	0.579	0.157	0.425	0.552
SVM + TF-IDF	0.575	0.562 (0.526–0.599)	0.565	0.135	0.560	0.542
source/sink rule (baseline)	0.548	0.480 (0.446–0.515)	0.524	0.063	0.796	0.476
static taint analyzer	0.541	0.366 (0.360–0.373)	0.504	0.035	0.982	0.465
majority (baseline)	0.538	0.350 (0.349–0.350)	0.500	0.000	1.000	0.462

4.3. Per-Class Behavior and Degenerate Predictors

Aggregate scores conceal how several models achieve them. Table 7 gives per-class F1 for the four-class Variant B setting, and Figure 3 shows representative confusion matrices.

Table 7. Per-class F1 (mean ± standard deviation over folds), Variant B, four-class. Class support: Safe 13,396; SQLi 2142; XSS 8531; RCE 848.

Model	Safe	SQLi	XSS	RCE
stacking (nested)	0.579 ± 0.008	0.071 ± 0.027	0.387 ± 0.017	0.085 ± 0.013
SVM + TF-IDF	0.641 ± 0.005	0.134 ± 0.098	0.362 ± 0.023	0.065 ± 0.024
Random Forest + TF-IDF	0.701 ± 0.007	0.004 ± 0.007	0.220 ± 0.024	0.000 ± 0.000
Bi-LSTM	0.413 ± 0.164	0.108 ± 0.077	0.203 ± 0.168	0.027 ± 0.033
uni-LSTM	0.416 ± 0.074	0.031 ± 0.051	0.500 ± 0.012	0.002 ± 0.004
combined	0.449 ± 0.111	0.146 ± 0.028	0.336 ± 0.134	0.008 ± 0.015
function length (baseline)	0.669 ± 0.003	0.032 ± 0.009	0.380 ± 0.032	0.053 ± 0.018
majority (baseline)	0.699 ± 0.000	0.000 ± 0.000	0.000 ± 0.000	0.000 ± 0.000

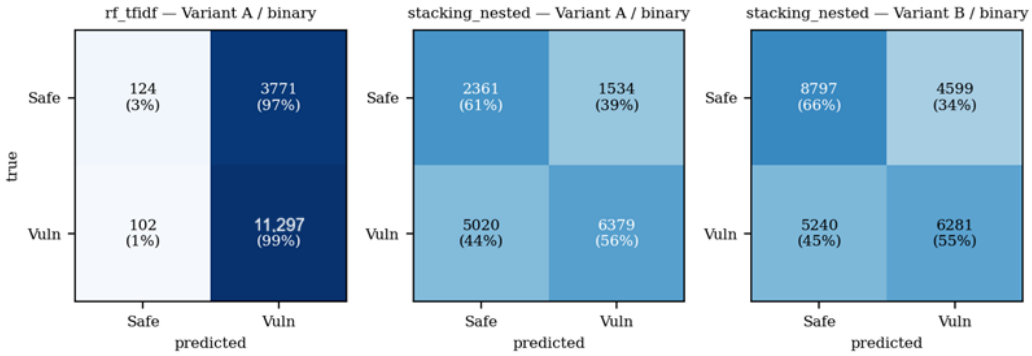


Figure 3. Confusion matrices summed over the five folds, so each held-out record is counted exactly once. Left: the TF-IDF Random Forest on Variant A binary, which assigns almost every record to the vulnerable class. Center and right: the nested stacking ensemble on Variants A and B. Darker shades indicate a higher number of observations, whereas lighter shades indicate a lower number of observations.

On Variant A the TF-IDF Random Forest labels 15,068 of 15,294 records as vulnerable, giving a Safe-class recall of 0.032 and a Safe F1 of 0.060. Its accuracy of 0.747 exceeds the majority baseline of 0.745 by 0.15 percentage points, and its macro-F1 advantage over that baseline is significant across folds but not across plugins (Wilcoxon $p = 0.295$, Cohen's $d = 0.057$). On Variant B the same model fails in the opposite direction, with a vulnerable-class false-negative rate of 0.84 to 0.87. The static taint analyzer detects 1.7 to 1.8% of vulnerabilities. Several models also predict the RCE class almost never: the Random Forest never predicts it at all, and the uni-LSTM attains an RCE F1 of 0.002 while reaching 0.500 on XSS. The nested stacking ensemble is the most balanced predictor, with a Safe F1 of 0.419 on Variant A binary against the Random Forest's 0.060.

4.4. RQ4: How Much Do Leakage and In-Sample Stacking Inflate Results?

Two sources of optimistic bias are quantified directly. Replacing the in-sample stacking implementation with nested out-of-fold feature generation changes macro-F1 as shown in Table 8.

Table 8. Optimistic bias of in-sample stacking. Positive values indicate that the in-sample implementation reports a better score than the honest nested one.

Configuration	In-Sample	Nested OOF	Bias (pp)
Variant A, binary	0.561	0.540	+2.17
Variant A, three-class	0.384	0.372	+1.15
Variant A, four-class	0.298	0.268	+2.95
Variant B, binary	0.585	0.601	−1.62
Variant B, three-class	0.401	0.398	+0.28
Variant B, four-class	0.307	0.281	+2.68

The mean bias is +1.27 percentage points, and the in-sample form is optimistic in five of six configurations, reaching +2.95. It is not uniformly optimistic: in Variant B binary, the nested model is 1.6 points better. The appropriate conclusion is that in-sample meta-feature generation is unreliable rather than that it inflates by a fixed amount. In Variant A binary, the in-sample form also wins on accuracy and macro-F1 while losing on balanced accuracy, MCC and PR-AUC, indicating that the leaked features bias the ensemble toward the majority class.

The second source, duplicate leakage, cannot be expressed as a single number because removing duplicates changes the corpus, its class balance, and therefore the baselines themselves. Its magnitude is instead given by Tables 1 and 2: before correction, 1086 and 676 records were exact copies of training material appearing in test folds, and 1852 and 1199 records carried labels contradicted by identical code elsewhere in the corpus.

4.5. Label Cleaning Revisited

The earlier version reported that restricting the corpus to functions with a complete source-to-sink path raises three-class macro-F1 by 10.4 percentage points and described this as the dominant improvement lever. Recomputing the comparison on the deduplicated corpora shows that the effect is real for most learned models but is confounded and far less uniform than previously stated (Table 9).

Table 9. Macro-F1 on the full corpus versus the source-to-sink filtered subset. The filtered subset is 15.7% of Variant A three-class (2266 of 14,445) and 14.5% of Variant B binary (3615 of 24,917).

Model	A 3-cl. Full	A 3-cl. Filtered	Δ	B bin. Full	B bin. Filtered	Δ
stacking (nested)	0.372	0.513	+0.141	0.601	0.612	+0.011
SVM + TF-IDF	0.377	0.490	+0.112	0.562	0.571	+0.009
Random Forest + TF-IDF	0.292	0.513	+0.222	0.586	0.599	+0.013
function length (baseline)	0.306	0.283	−0.023	0.572	0.594	+0.022
static taint analyzer	0.160	0.290	+0.129	0.366	0.451	+0.085
source/sink rule (baseline)	0.198	0.078	−0.120	0.480	0.297	−0.183
majority (baseline)	0.246	0.228	−0.018	0.350	0.366	+0.016

Two caveats must accompany these numbers, and the first also applied to the previously published comparison without being stated there. The full and filtered corpora are different test sets, and the filter retains textbook source-to-sink functions, so the difference mixes improved label quality with an easier subset. Consistent with that, the gain is largest for the weakest models, the function-length baseline becomes worse on the filtered subset in three of four settings, and the source/sink baseline collapses because the filter selects precisely the records on which it always fires. In Variant B binary, the gain for learned models is between 0.9 and 2.2 percentage points, that is, close to nothing. Second, collapsing clone families breaks the before/after pairing, so the filter is applied per record rather than per pair. Label cleaning therefore remains a plausible but unproven lever, and we no longer present it as the dominant one.

4.6. Expert Validation of Labels

Because label quality is central to the interpretation, it was measured directly rather than inferred. A stratified sample of 400 functions was drawn across the three confidence tiers produced by CVE-anchored scoring: all 37 HIGH pairs (source and sink present and confirmed by the CVE description), 217 MEDIUM pairs (source and sink present, unconfirmed by text) and 146 LOW pairs (no single-function source-to-sink path). A human analyst labeled each function as a genuine vulnerability, an automated-label error on non-vulnerable code, diff-induced noise, or an ambiguous case, without access to the automated verdict. The reliability of the automated labeling procedure was further assessed through expert validation using a stratified sample of 400 functions. The results of this manual validation, including the precision of the automatically assigned vulnerability labels, are presented in Table 10.

Table 10. Expert validation of automated labels on a stratified 400-function sample. Precision is the fraction judged genuinely vulnerable by a human analyst.

Confidence Tier	n	Correct	Precision	False Positive	Noisy	Unclear
HIGH	37	27	73.0%	2	2	6
MEDIUM	217	97	44.7%	38	71	11
LOW (discarded)	146	1	0.7%	75	59	11
HIGH + MEDIUM (cleaned subset)	254	124	48.8%	40	73	17

The cleaned positive labels are correct in only 48.8% of cases, while the discarded LOW tier contains almost no genuine vulnerabilities (0.7%), indicating that the cleaning filter removes non-vulnerable code rather than real flaws. The verified set comprises 125 confirmed vulnerabilities (85 XSS, 40 SQLi) and 247 confirmed-safe functions and is released as a high-confidence evaluation subset. The sample was labeled by a single annotator, so no inter-annotator agreement can be reported; this is a limitation, and a larger sample with at least two annotators and adjudication is the natural next step.

4.7. Inter-Procedural Recovery

A separate experiment tested whether functions discarded by the source-to-sink filter merely placed the source and the sink in different functions of the same file. Table 11 reports the reconciled counts; the previously published version omitted two rows, which made the arithmetic appear not to close.

Table 11. Inter-procedural recovery of discarded pairs. The categories sum to 13,702. Thirty-nine inter-procedural paths were found; after deduplication against the cleaned set these correspond to 38 unique new pairs (76 records), which is why both figures appear in the analysis and why the cleaned corpus grows from 3456 to 3532 records.

Category	Pairs
Noisy pairs analyzed	13,702
No source token in the function	9726
Source present, no local callees	50
Plugin file not found in cache	0
Path found—intra-procedural duplicate of the cleaned set	153
Path found—inter-procedural, depth 1	37
Path found—inter-procedural, depth 2	2
Not recovered	3734

Only 0.3% of the discarded pairs contain a genuine inter-procedural path, and 71% contain no taint source at all. The functions removed by the filter are therefore mostly non-vulnerable rather than vulnerable in a way the analysis failed to see, which is consistent with the 0.7% false-negative rate measured by expert review.

4.8. Qualitative Error Analysis

False negatives were sampled only from records whose ground truth was confirmed by the manual annotation, so that each example is a genuine miss rather than a labeling artifact. On Variant B binary, the TF-IDF Random Forest has 39 expert-confirmed false negatives and the Bi-LSTM has 14. The most confident misses are an unsanitized POST parameter written to an option and echoed back in the IMPress for IDX Broker plugin (CVE-2020-11512, predicted Safe with probability 0.743) and an unescaped plugin-name comparison in Profile Builder (CVE-2015-9328, probability 0.680) for the Random Forest; and an XSS in the Events Manager booking administration (CVE-2015-9299, probability 0.653) and an unparameterised DELETE built from a POST field in Booking System (CVE-2014-3210, probability 0.640) for the Bi-LSTM. In each case, the vulnerable pattern is lexically unremarkable, which is consistent with the models relying on coarse surface features. To contextualize the performance of the proposed approach, its results were compared with those of representative learning-based vulnerability detectors reported in previous studies. Because the compared approaches differ in programming language, dataset composition, and, in particular, evaluation protocols and data splits, the reported performance values should be interpreted as indicative rather than directly comparable. The comparison is presented in Table 12.

Table 12. Comparison with representative learning-based detectors. Differences in language, dataset and especially evaluation split make absolute numbers only indirectly comparable.

Study	Year	Lang.	Task	Eval. Split	Metric	Value
LineVul [10]	2022	C/C++	Binary (Big-Vul)	Random	F1	0.91
DeepTective [11]	2022	PHP	Multi (SQLi/XSS/CI)	Random	F1	0.881
CodeBERT (CodeXGLUE) [12,14]	2020	C	Binary (Devign)	Fixed	Acc.	0.621
This work (best)	2026	PHP	Binary (WP CVE)	Plugin-grouped, deduplicated	macro-F1	0.601

5. Discussion

5.1. Comparison with Reported Results

The gap between our figures and the higher numbers reported elsewhere is best explained by methodology rather than model quality. Random splits allow near-duplicate functions to appear in both training and test, which our audit shows is not a hypothetical concern in this domain: even after grouping by plugin, 1086 records in Variant A were exact copies of training material. Our labels are derived from security-fix diffs and, as the expert validation shows, are correct for only about half of the positive class. Both factors push measured performance down relative to curated corpora, and both are properties of the data rather than of the detectors.

Taken together, the results suggest a reordering of priorities. The spread between the best and worst learned models within a setting is typically smaller than the spread introduced by changing the negative-sampling variant, and a single-feature baseline sits within that spread. Architecture and representation choices therefore appear to be second-order in this domain, while label quality—measured at 48.8% precision on the cleaned positive class—is the more plausible binding constraint. We state this as the most consistent

reading of the evidence rather than as a demonstrated causal claim, since the label-cleaning comparison that would establish it directly is confounded by subset difficulty (Section 4.5).

5.2. Threats to Validity

The protocol is duplicate-controlled rather than provably leakage-free. Exact duplicates and token- and AST-level clone families are removed, and the released folds share no hash, clone family or plugin, but semantic clones that differ lexically and structurally may persist. Collapsing clone families also changes the class balance, most sharply in Variant A, where the majority class becomes the vulnerable class; accuracy is therefore not comparable with the earlier release and macro-F1, balanced accuracy and MCC should be used for comparison.

The dataset is incomplete: about 54% of indexed CVEs were unrecoverable because pre-fix releases and old SVN tags were pruned, so the corpus may be biased toward plugins that retain historical versions. Rare-class estimates are unreliable—RCE has 848 records in Variant B and several models never predict it—so four-class results should be treated as indicative. The neural models were trained on at most 4000 records per fold for 40 epochs under a CPU budget, so their numbers are a lower bound; the test folds, however, are always complete. A transformer comparison is absent: the transformers library is not installed in the evaluation environment and the host has no GPU, so CodeBERT was not run and is not reported. A 256-token sensitivity run for the sequence models is likewise outstanding. Finally, the taint analyzer is intra-procedural and lexical, so its low recall reflects that implementation rather than taint analysis in general.

6. Limitations and Future Work

The central limitation is that the labels themselves are only about half correct on the positive class, which bounds what any model trained on them can demonstrate. The priority is therefore data rather than architecture. First, expert-verified data at scale: extending the 400-function review to several thousand functions with at least two annotators, adjudication and reported agreement, and evaluating on a genuinely expert-verified test set. Second, curated vulnerability feeds the supply entry point and sink explicitly, reducing reliance on diff heuristics. Third, semantic clone detection beyond token and AST similarity, since the present analysis removes only lexical and structural duplicates. Fourth, the outstanding runs: a fully resourced transformer comparison on identical folds and the 256-token sensitivity analysis. Fifth, given that a single-feature length baseline is competitive, future detectors in this domain should be required to clear such trivial baselines before semantic detection is claimed.

7. Conclusions

We presented a duplicate-controlled benchmark of real WordPress plugin vulnerabilities and re-evaluated ten detectors on it. An audit of our own first release showed that plugin-level grouping does not prevent exact code duplicates from crossing fold boundaries, that identical code sometimes carries contradictory labels, and that a stacking ensemble described as out-of-fold was trained in sample. After correcting all three, the corpora contain 15,294 and 24,917 records with verified zero overlap across folds.

On these corpora the best macro-F1 is 0.563 and 0.601 in the binary settings. No neural configuration leads in any of the six settings, a single-feature function-length classifier matches or exceeds the TF-IDF Random Forest in four of six, the source/sink rule has essentially no standalone detection ability, and the static analyzer misses about 98% of vulnerabilities. In-sample stacking is optimistic by 1.27 percentage points on average. Expert review finds 48.8% of the cleaned positive labels to be genuine.

The practical implication is that in this domain the choice of detector architecture matters less than the quality of the supervision and the rigor of the evaluation protocol. We release the corpora, folds, per-fold predictions and scores, and fingerprinted result files so that each of these findings can be checked and extended.

Author Contributions: Conceptualization, Z.T. and A.A.; methodology, A.Z. and A.A.; software, A.A. and S.A.; validation, S.A. and A.A.; formal analysis, A.U. and A.A.; investigation, Z.T. and A.A.; resources, A.A.; data curation, A.K.; writing—original draft preparation, S.A. and A.A.; writing—review and editing, A.Z. and A.A.; visualization, A.U. and A.A.; supervision, Z.T.; project administration, A.K. and A.A. All authors have read and agreed to the published version of the manuscript.

Funding: This research received no external funding.

Institutional Review Board Statement: Not applicable.

Informed Consent Statement: Not applicable.

Data Availability Statement: The original data presented in the study are openly available in the wordpress-cve-benchmark repository on GitHub at https://github.com/Aisultan05/wordpress-cve-benchmark (accessed on 2 July 2026) and archived in Zenodo at https://doi.org/10.5281/zenodo.21240149 (accessed on 2 July 2026). The audited rebuild described in Sections 3.2 and 4 is available as release v2.1.0 of the repository and is archived in Zenodo at https://doi.org/10.5281/zenodo.21931697. To respect the licenses of third-party plugins, the repository provides labels, metadata, hashes, and a loader script that reconstructs the function snippets locally from the official WordPress.org repository; plugin source code is not redistributed.

Acknowledgments: The author thanks the anonymous reviewers, whose audit of the released artifacts identified the duplication, labeling and stacking problems corrected in this version.

Conflicts of Interest: The author declares no conflicts of interest.

References

1. Demir, N.; Urban, T.; Wittek, K.; Pohlmann, N. Our (In)Secure Web: Understanding Update Behavior of Websites and Its Impact on Security. In *Passive and Active Measurement (PAM 2021), LNCS 12671*; Springer: Cham, Switzerland, 2021; pp. 76–92. [CrossRef]
2. Patchstack. *State of WordPress Security in 2024: Annual Report*; Patchstack: Pärnu, Estonia, 2024.
3. OWASP Foundation. OWASP Top 10: 2021. Available online: https://owasp.org/Top10/2021/ (accessed on 31 July 2026).
4. Tashenova, Z.; Aitmagambetuly, A.; Bayegizova, A.; Santeyeva, S.; Abdugulova, Z.; Amanzholova, S.; Kerim, A. SentinelCMS: Proactive Vulnerability Detection in CMS Plugins Using Static Taint Analysis and Bidirectional LSTM. *Appl. Sci.* **2026**, *16*, 5471. [CrossRef]
5. Ekstam Ljusegren, H. Vulnerabilities in Outdated Content Management Systems: An Analysis of the Largest WordPress Websites. Master's Thesis, Linköping University, Linköping, Sweden, 2023.
6. Jovanovic, N.; Kruegel, C.; Kirda, E. Pixy: A Static Analysis Tool for Detecting Web Application Vulnerabilities. In Proceedings of the IEEE S&P, Berkeley/Oakland, CA, USA, 21–24 May 2006; pp. 258–263.
7. Yamaguchi, F.; Golde, N.; Arp, D.; Rieck, K. Modeling and Discovering Vulnerabilities with Code Property Graphs. In Proceedings of the IEEE S&P, San Jose, CA, USA, 18–21 May 2014; pp. 590–604.
8. Zou, D.; Wang, S.; Xu, S.; Li, Z.; Jin, H. VulDeePecker: A Deep Learning-Based System for Vulnerability Detection. In Proceedings of the NDSS, San Diego, CA, USA, 18–21 February 2018.
9. Zhou, Y.; Liu, S.; Siow, J.; Du, X.; Liu, Y. Devign: Effective Vulnerability Identification by Learning Comprehensive Program Semantics via Graph Neural Networks. In Proceedings of the NeurIPS, Vancouver, BC, Canada, 8–14 December 2019.
10. Fu, M.; Tantithamthavorn, C. LineVul: A Transformer-based Line-Level Vulnerability Prediction. In Proceedings of the MSR, Pittsburgh, PA, USA, 23–24 May 2022.
11. Rabheru, R.; Hanif, H.; Maffeis, S. A Hybrid Graph Neural Network Approach for Detecting PHP Vulnerabilities. In Proceedings of the 2022 IEEE Conference on Dependable and Secure Computing (DSC), Edinburgh, UK, 22–24 June 2022; pp. 1–9. [CrossRef]
12. Feng, Z.; Guo, D.; Tang, D.; Duan, N.; Feng, X.; Gong, M.; Shou, L.; Qin, B.; Liu, T.; Jiang, D.; et al. CodeBERT: A Pre-Trained Model for Programming and Natural Languages. In *Findings of the Association for Computational Linguistics: EMNLP 2020*; Cohn, T., He, Y., Liu, Y., Eds.; Association for Computational Linguistics: Stroudsburg, PA, USA, 2020; pp. 1536–1547. [CrossRef]

13. Guo, D.; Ren, S.; Lu, S.; Feng, Z.; Tang, D.; Liu, S.; Zhou, L.; Duan, N.; Svyatkovskiy, A.; Fu, S.; et al. GraphCodeBERT: Pre-training Code Representations with Data Flow. In Proceedings of the ICLR, Vienna, Austria, 3–7 May 2021.
14. Zhou, X.; Cao, S.; Sun, X.; Lo, D. Large Language Model for Vulnerability Detection and Repair: Literature Review and the Road Ahead. *ACM Trans. Softw. Eng. Methodol.* **2025**, *34*, 129. [CrossRef]
15. Lu, S.; Guo, D.; Ren, S.; Huang, J.; Svyatkovskiy, A.; Blanco, A.; Clement, C.; Drain, D.; Jiang, D.; Tang, D.; et al. CodeXGLUE: A Machine Learning Benchmark Dataset for Code Understanding and Generation. In Proceedings of the NeurIPS Datasets and Benchmarks, Vancouver, BC, Canada, 6–14 February 2021.
16. Chakraborty, S.; Krishna, R.; Ding, Y.; Ray, B. Deep Learning Based Vulnerability Detection: Are We There Yet? *IEEE Trans. Softw. Eng.* **2022**, *48*, 3280–3296. [CrossRef]
17. Arp, D.; Quiring, E.; Pendlebury, F.; Warnecke, A.; Pierazzi, F.; Wressnegger, C.; Cavallaro, L.; Rieck, K. Dos and Don'ts of Machine Learning in Computer Security. In Proceedings of the 31st USENIX Security Symposium, Boston, MA, USA, 10–12 August 2022; pp. 3971–3988.
18. Croft, R.; Babar, M.A.; Kholoosi, M.M. Data Quality for Software Vulnerability Datasets. In Proceedings of the 45th IEEE/ACM International Conference on Software Engineering (ICSE 2023), Melbourne, Australia, 14–20 May 2023; pp. 121–133. [CrossRef]
19. Ding, Y.; Fu, Y.; Ibrahim, O.; Sitawarin, C.; Chen, X.; Alomair, B.; Wagner, D.; Ray, B.; Chen, Y. Vulnerability Detection with Code Language Models: How Far Are We? In Proceedings of the 47th IEEE/ACM International Conference on Software Engineering (ICSE 2025), Ottawa, ON, Canada, 27 April–3 May 2025; pp. 1729–1741. [CrossRef]
20. National Vulnerability Database (NVD). WordPress Plugin CVE Records. Available online: https://nvd.nist.gov/ (accessed on 2 July 2026).
21. CVE Program. CVE List V5; CVEProject. Available online: https://github.com/CVEProject/cvelistV5 (accessed on 2 July 2026).

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.